

Data Analysis Project: Diabetes Prediction with Bayesian

Sandipan Dey

Executive summary

In this project, we shall try to understand how diabetes is dependent on a given a set of diagnostic measurements. More specifically, our objective will be:

1. Use the diagnostic measurements as predictor variables and *estimate* the coefficients for the predictors under Bayesian settings
2. *Predict* the response variable, i.e., whether a patient is *diabetic* or not, given those measurements and computed coefficients
3. Construct *Bayesian Credible Intervals (CI)* for the coefficients to quantify uncertainty and make probabilistic claims on the coefficients.
4. Compute the *Posterior Predictive Distribution* and use it to compute *CI* for the predictions and make probabilistic claims on the predictions for different patients.
5. We shall use base R and rjags library functions for the implementation.

Introduction

Predicting whether a patient is diabetic from her medical diagnosis will be the goal of this project. Under Bayesian settings, we shall impose *priors* on the coefficients corresponding to the predictors (diagnostic measurements) and use *MCMC* sampling to compute the *posterior* distributions, verifying that the Markov chains converge to the target stationary distribution with *diagnostic tests*. We shall try to fit a few different models and use *model checking / selection* to find the best model (with *DIC*), along with that we shall use *posterior predictive distribution* to answer to a few questions (test *hypotheses*) related to prediction of the outcomes, we shall also construct *Bayesian credible intervals* to quantify the uncertainty for the coefficient estimates and also for our prediction. As we shall see, few of the predictor variables we shall find as statistically significant.

Data

We shall use the Diabetes dataset from Kaggle: <https://www.kaggle.com/datasets/mathchi/diabetes-data-set> (<https://www.kaggle.com/datasets/mathchi/diabetes-data-set>). This dataset is originally from the National Institute of Diabetes and Digestive and Kidney Diseases. Several constraints were placed on the selection of these instances from a larger database. In particular, all patients here are females at least 21 years old of Pima Indian heritage. Each row corresponds to the measurements corresponding to a patient (there are 768 rows in the dataset), containing 8 **predictor** variables listed and described below (which can be used to predict the response variable), with a single **response** variable named *Outcome* (which is 1 if the patient is diabetic, 0 otherwise).

- Pregnancies: Number of times pregnant
- Glucose: Plasma glucose concentration a 2 hours in an oral glucose tolerance test
- BloodPressure: Diastolic blood pressure (mm Hg)
- SkinThickness: Triceps skin fold thickness (mm)
- Insulin: 2-Hour serum insulin (mu U/ml)
- BMI: Body mass index (weight in kg/(height in m)²)
- DiabetesPedigreeFunction: Diabetes pedigree function

- Age: Age (years)
- Outcome: Class variable (0 or 1)

The following table shows first few rows of the dataset.

```
## [1] 768 9
```

```
## Pregnancies Glucose BloodPressure SkinThickness Insulin BMI
## 1          6      148           72           35      0 33.6
## 2          1       85           66           29      0 26.6
## 3          8     183           64            0      0 23.3
## 4          1       89           66           23     94 28.1
## 5          0     137           40           35    168 43.1
## 6          5     116           74            0      0 25.6

## DiabetesPedigreeFunction Age Outcome
## 1          0.627  50      1
## 2          0.351  31      0
## 3          0.672  32      1
## 4          0.167  21      0
## 5          2.288  33      1
## 6          0.201  30      0
```

Exploratory Analysis & Cleaning

- First we omit the *NA* values, if any, from the dataset.
- We observe that many of the predictors (e.g., *BloodPressure*, *SkinThickness*, *Insulin*) contain quite a few *zero* values (which appears to be impossible value for the measurements, hence), we suspect they are noise.
- We must remove / replace the noise / outlier values, otherwise the model fitted can be very inaccurate.
- We first compute what percentage of values for a predictor contain inappropriate *0* values and find that there are quite significant (as shown below), so we can't drop the corresponding observations, since they will reduce the data size a lot.
- Instead, we use mean imputation technique to replace the *zero* values for a predictor with the *mean* of its non-zero values, as shown below.
- Also, let's verify there is no remainign zero values in the predicator variables after imputation, so that we get clean data (only the *Outcome* column which is the response variable can contain zero values).

```
## [1] "percent zero values for each predictor variable columns"
```

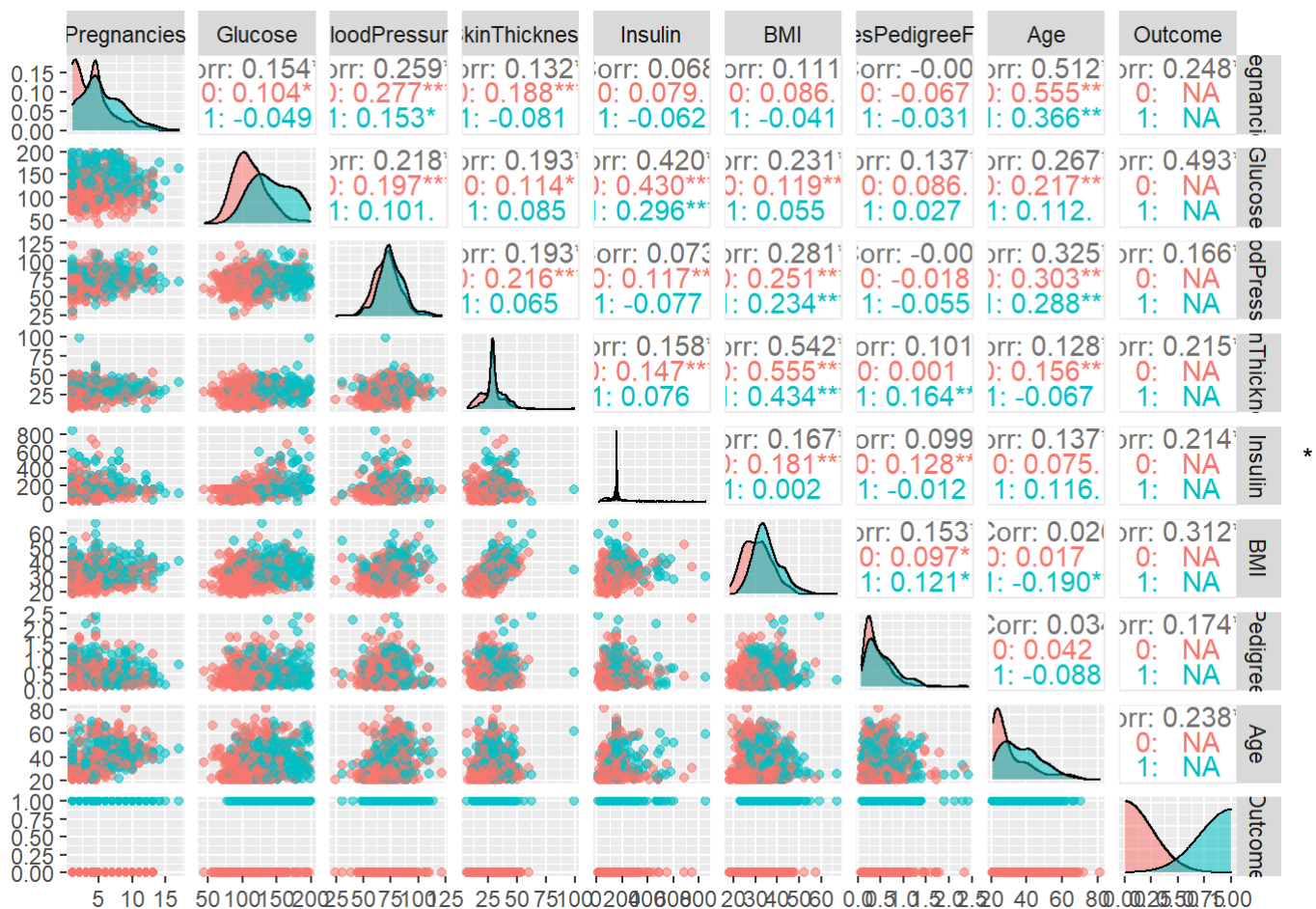
```
## Pregnancies      Glucose      BloodPressure
## 14.4531250      0.6510417      4.5572917
## SkinThickness      Insulin      BMI
## 29.5572917      48.6979167      1.4322917
## DiabetesPedigreeFunction Age      Outcome
## 0.0000000      0.0000000      65.1041667
```

```
## [1] "percent zero values for each predictor variable columns after mean imputation"
```

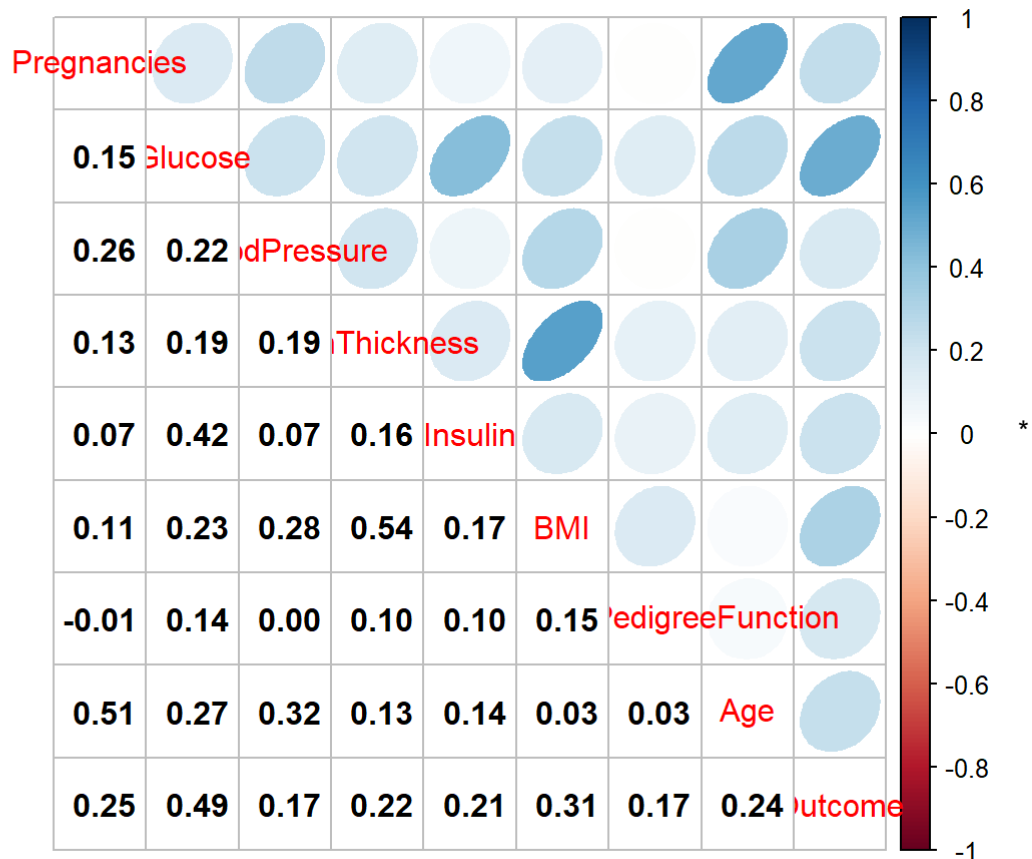
##	Pregnancies	Glucose	BloodPressure
##	0.00000	0.00000	0.00000
##	SkinThickness	Insulin	BMI
##	0.00000	0.00000	0.00000
##	DiabetesPedigreeFunction	Age	Outcome
##	0.00000	0.00000	65.10417

- Now, let's compute the summary of the dataset and visualize the correlation between the variables (separately for *diabetes* and *non-diabetes* observations, denoted by the colors red and blue, respectively), as shown below.

##	Pregnancies	Glucose	BloodPressure	SkinThickness
##	Min. : 1.000	Min. : 44.00	Min. : 24.00	Min. : 7.00
##	1st Qu.: 2.000	1st Qu.: 99.75	1st Qu.: 64.00	1st Qu.:25.00
##	Median : 4.495	Median :117.00	Median : 72.20	Median :29.15
##	Mean : 4.495	Mean :121.69	Mean : 72.41	Mean :29.15
##	3rd Qu.: 6.000	3rd Qu.:140.25	3rd Qu.: 80.00	3rd Qu.:32.00
##	Max. :17.000	Max. :199.00	Max. :122.00	Max. :99.00
##	Insulin	BMI	DiabetesPedigreeFunction	Age
##	Min. : 14.0	Min. :18.20	Min. :0.0780	Min. :21.00
##	1st Qu.:121.5	1st Qu.:27.50	1st Qu.:0.2437	1st Qu.:24.00
##	Median :155.5	Median :32.40	Median :0.3725	Median :29.00
##	Mean :155.5	Mean :32.46	Mean :0.4719	Mean :33.24
##	3rd Qu.:155.5	3rd Qu.:36.60	3rd Qu.:0.6262	3rd Qu.:41.00
##	Max. :846.0	Max. :67.10	Max. :2.4200	Max. :81.00
##	Outcome			
##	Min. :0.000			
##	1st Qu.:0.000			
##	Median :0.000			
##	Mean :0.349			
##	3rd Qu.:1.000			
##	Max. :1.000			

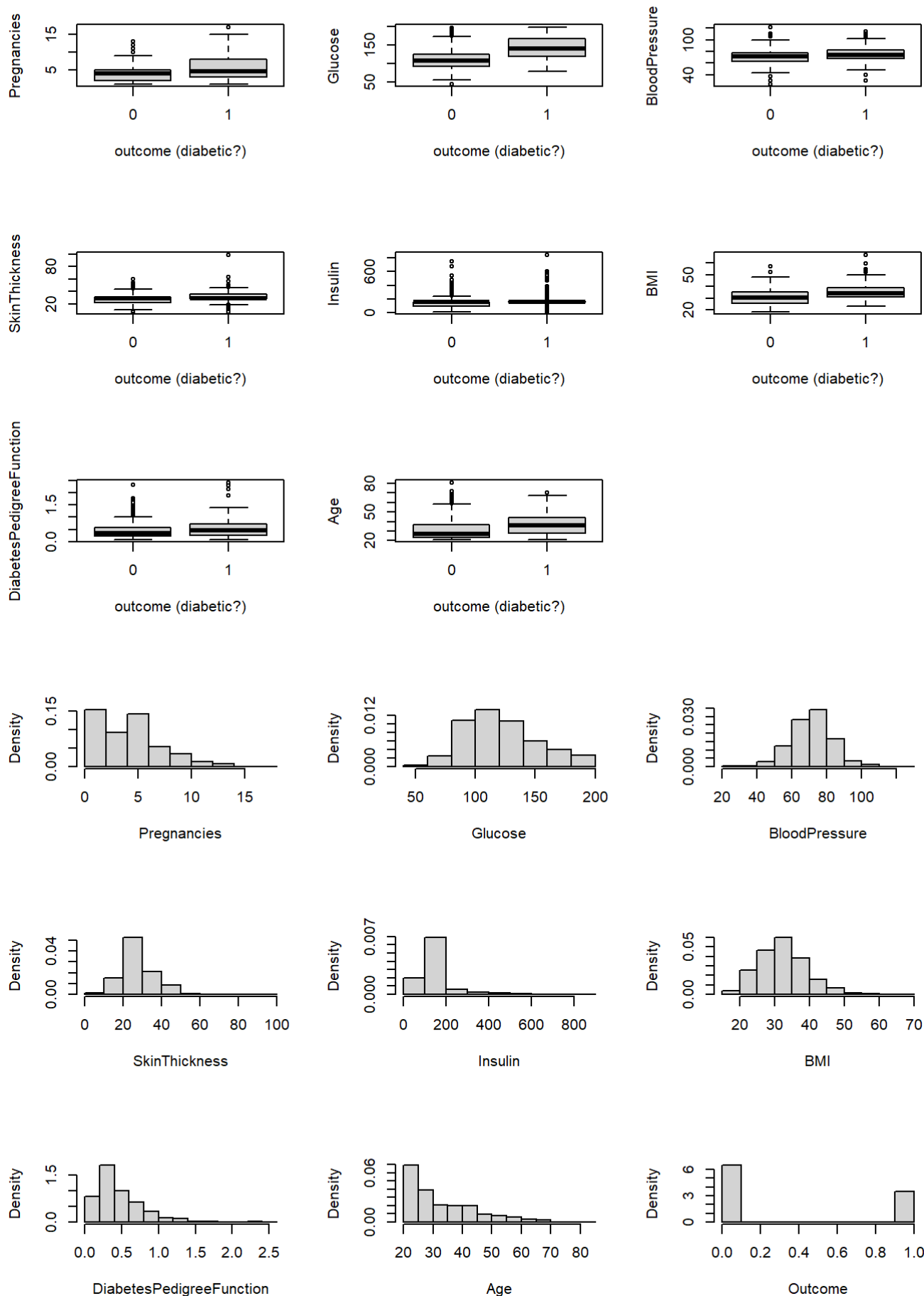


Also, let's plot the correlation between the predictors and the outcome, with darker color representing higher correlation value.



Next let's find out whether the distributions of the predictors given *Outcome*=0 and *Outcome*=1 differ significantly using the following *boxplot*. Also, let's visualize the predictor distributions from the data using

histograms (we can see some of the predictors in the data, e.g. *Age* are very much *right-skewed*). As can be seen from below, the distribution of the predictor variable *SkinThickness* is almost same when conditioned on different values of the *outcome*, so it's likely not to be a good predictor.



Model

- We shall first fit the *glm* (generalized linear model) in R with `family=binomial`, with `Outcome` as the *response* and the rest of the variables as predictors. Plot the residuals (diagnostic plot) to see if there is any pattern and whether the model assumptions are met. Note the values of the coefficients computed, the statistically significant ones. Finally, predict using the model fitted and compute the prediction *accuracy* along with the *confusion matrix*.
- Then, we shall use `rjags` to fit Bayesian models. Let's first *z-score normalize (scale)* the predictors and verify that each of them have mean *zero* and *unit* standard deviation.

```
## [1] "means (rounded to 3 decimals)"
```

```
##           Pregnancies           Glucose           BloodPressure
##                0                0                0
##       SkinThickness           Insulin           BMI
##                0                0                0
## DiabetesPedigreeFunction           Age
##                0                0
```

```
## [1] "standard deviations"
```

```
##           Pregnancies           Glucose           BloodPressure
##                1                1                1
##       SkinThickness           Insulin           BMI
##                1                1                1
## DiabetesPedigreeFunction           Age
##                1                1
```

- First, let's fit the following model with *Bernoulli* likelihood with *logit link* function and Gaussian prior (with zero mean, sd 5) on the intercept, along with *Laplace* (double exponential) prior (with zero mean and unit variance) on the remaining coefficients, to find posterior with sparse coefficients.

• **likelihood:** $y_i | x_i, \beta \stackrel{\text{ind}}{\sim} \text{Bern} \left(\frac{1}{1 + \exp[-(\beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \beta_3 x_{3,i})]} \right)$

• **priors:** $\beta_0 \sim N(0, 5^2)$, $\beta_i \sim L(0, 1/\sqrt{2})$
intercept $i \geq 1$

* We shall use 3 independent

Markov chains for mixing, reject the first 1000 samples from the *burn-in* period and simulate for another 5000 iterations, use *trace plot* to check the convergence (autocorrelation) as diagnostic.

- Also, in order to examine convergence of the *Markov chains* to *stationary* (target posterior) distribution, we shall use the *Gelman-Brooks-Rubin* diagnostic to verify if the Markov chains don't differ much / end in the same state-space after convergence (also check the effective sample size).
- We shall use the *Deviance information criterion (DIC)* for model selection (the lower the value, the better the model fit) and to compute the effective parameter size of the problem.
- We shall see that the posterior distribution of a few coefficients includes zero, so they are likely to be less important as predictors, we shall get rid of them and refit the model, this time with Gaussian prior on all the coefficients.

- **likelihood:** $y_i \mid x_i, \beta \stackrel{\text{ind}}{\sim} \text{Bern} \left(\frac{1}{1 + \exp[-(\beta_0 + \beta_1 x_{1,i} + \beta_2 x_{2,i} + \beta_3 x_{3,i})]} \right)$
- **priors:** $\beta \sim N(0, 5^2)$

JAGs model2

- Finally, we shall fit a third model using all the eight predictors and placing *flat normal* priors on them. We shall compare the models using *DIC* and select the best model.
- We shall compute the posterior mean of the *MCMC* samples (combined) to compute the estimate of the coefficients, also compute the *95% Highest Posterior Density (HPD)* credible intervals for the coefficients.
- We shall use the posterior estimates for the coefficients for prediction of the observations and compute the accuracy of prediction.
- Finally, we shall use the *posterior predictive distribution* to predict the probability of diabetes of couple of new patients, construct *95% Bayesian credible intervals (CI)* and compute the probability that one patient will have more chance to be diabetic over the other.

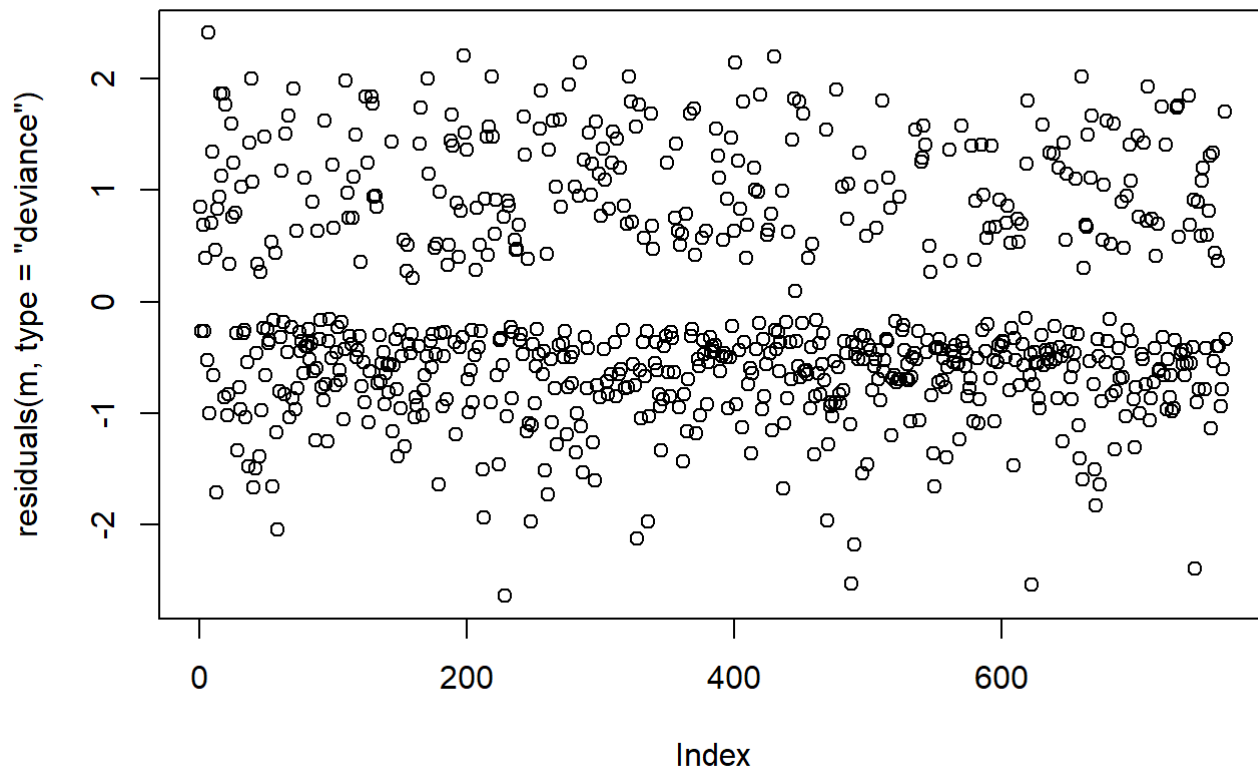
Results

Logistic Regression with GLM

As can be seen from the following results,

- The predictors `Pregnancies`, `Glucose`, `BMI`, `DiabetesPedigreeFunction` and `BloodPressure` are statistically significant (with *p-value* < 0.05, the null hypothesis can be rejected at 95% level of significance).
- The residual plot looks okay and does not show any pattern.
- Interpretation of the coefficients: e.g., increase in `Glucose` by 1 unit increases the *log odds* for diabetes by 0.0351637, whereas decrease in `BloodPressure` by 1 unit increases the *log odds* for diabetes by 0.0132955.
- The prediction has accuracy of 78.26%.

```
##
## Call:
## glm(formula = Outcome ~ ., family = "binomial", data = dat)
##
## Deviance Residuals:
##      Min       1Q   Median       3Q      Max
## -2.6353  -0.7129  -0.3951   0.6932   2.4061
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -9.0705975   0.8148288  -11.132 < 2e-16 ***
## Pregnancies     0.1406610   0.0359271   3.915 9.03e-05 ***
## Glucose         0.0372466   0.0038779   9.605 < 2e-16 ***
## BloodPressure  -0.0104619   0.0086383  -1.211 0.22585
## SkinThickness  0.0038213   0.0131661   0.290 0.77164
## Insulin        -0.0007215   0.0011808  -0.611 0.54115
## BMI            0.0890451   0.0177613   5.013 5.35e-07 ***
## DiabetesPedigreeFunction 0.8541331  0.2981939   2.864 0.00418 **
## Age           0.0152995   0.0091727   1.668 0.09533 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 993.48  on 767  degrees of freedom
## Residual deviance: 712.94  on 759  degrees of freedom
## AIC: 730.94
##
## Number of Fisher Scoring iterations: 5
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 443  57
##           1 114 154
##
##           Accuracy : 0.7773
##           95% CI : (0.7462, 0.8063)
##    No Information Rate : 0.7253
##    P-Value [Acc > NIR] : 0.0005657
##
##           Kappa : 0.4845
##
##    McNemar's Test P-Value : 1.849e-05
##
##           Sensitivity : 0.7953
##           Specificity : 0.7299
##           Pos Pred Value : 0.8860
##           Neg Pred Value : 0.5746
##           Prevalence : 0.7253
##           Detection Rate : 0.5768
##    Detection Prevalence : 0.6510
##           Balanced Accuracy : 0.7626
##
##           'Positive' Class : 0
##
```

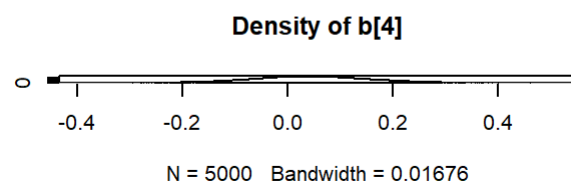
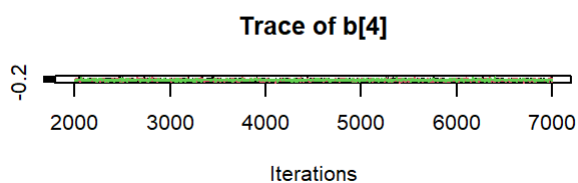
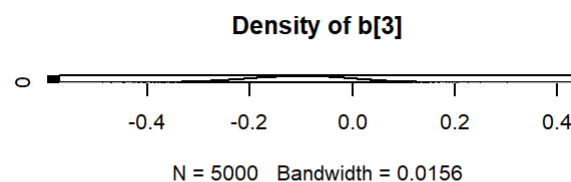
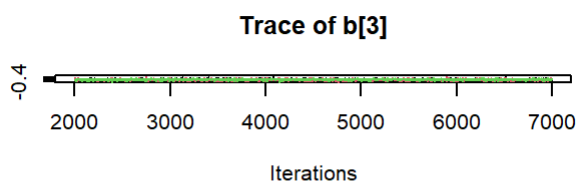
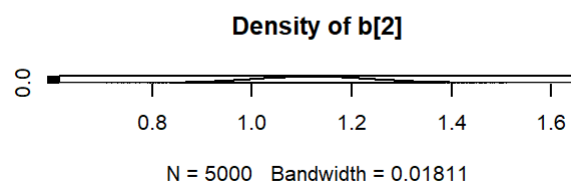
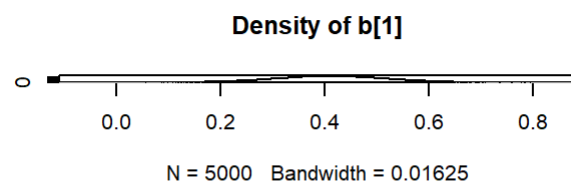
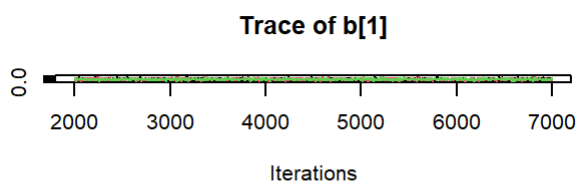
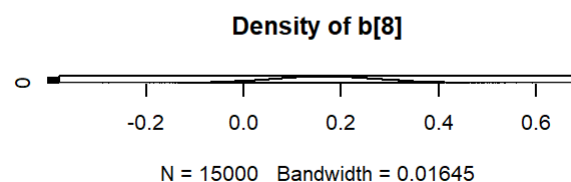
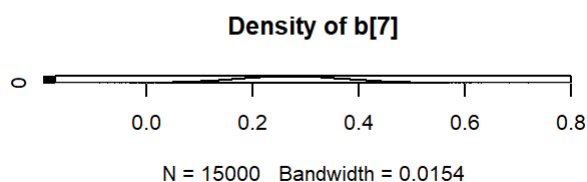
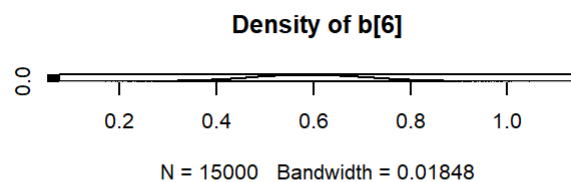
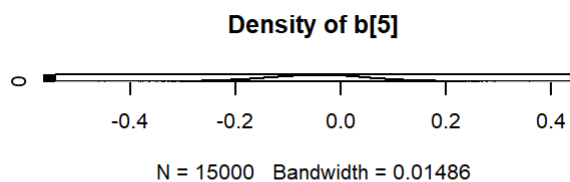
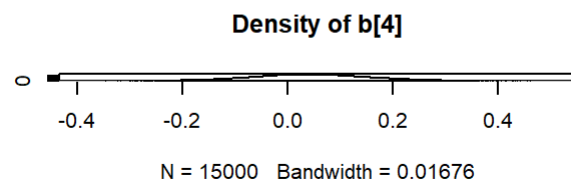
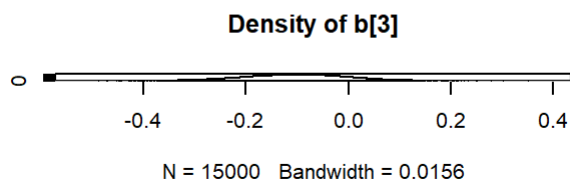
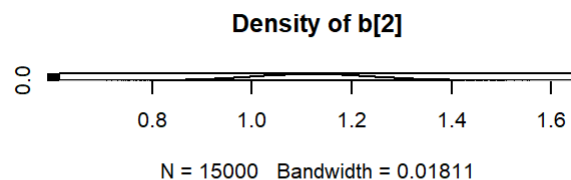
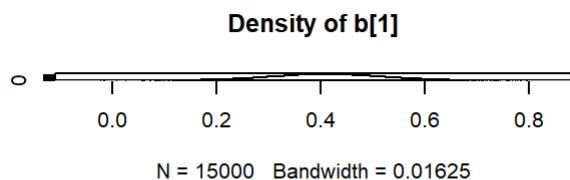
Bayesian Logistic Regression with RJags

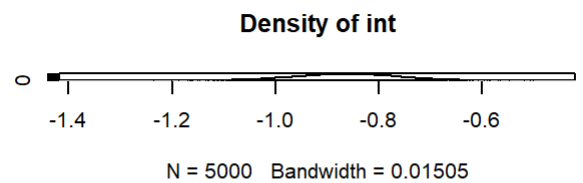
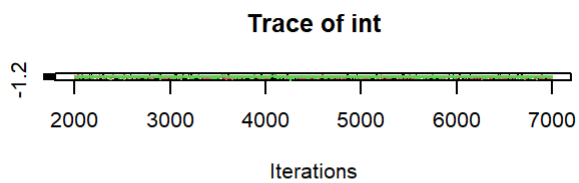
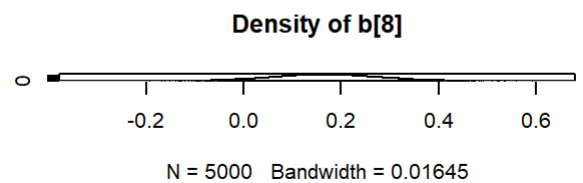
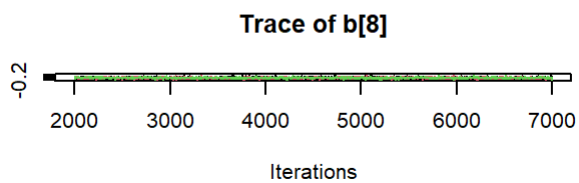
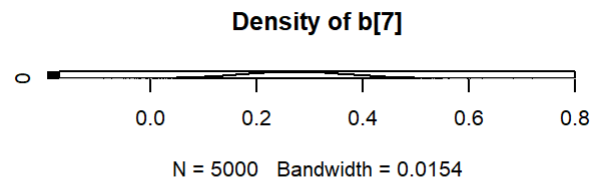
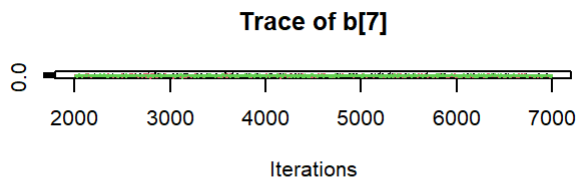
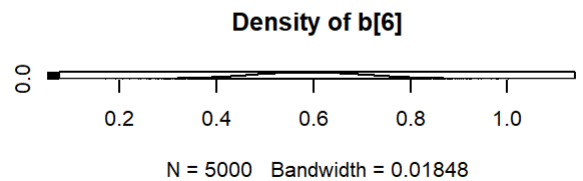
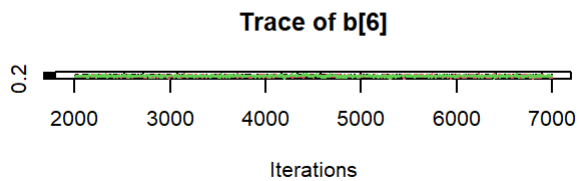
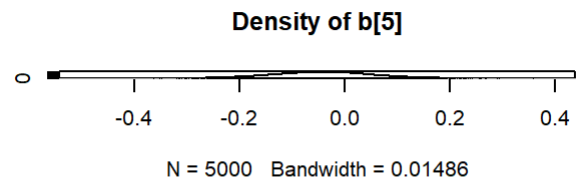
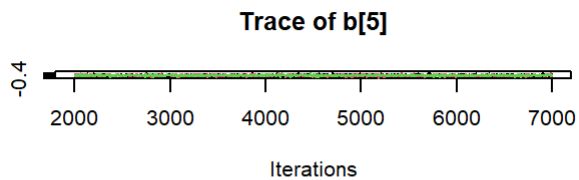
Model 1

For the model with *double exponential* (*Laplace*) prior on the coefficients,

- The trace plots for the MCMC looks okay, indicating convergence.
- The Gelman diagnostic looks okay too, the values are close to 1, as expected.
- The coefficients for `SkinThickness`, `Insulin` and `Age` includes zero in posterior distribution, we can ignore them.
- The DIC for the model is 740.7

```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 768
##   Unobserved stochastic nodes: 9
##   Total graph size: 9732
##
## Initializing model
```





```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## b[1]  0.41003 0.10547 0.0008612      0.001426
## b[2]  1.12541 0.11687 0.0009543      0.001494
## b[3] -0.10410 0.10069 0.0008221      0.001218
## b[4]  0.04352 0.10933 0.0008927      0.001436
## b[5] -0.04339 0.09641 0.0007872      0.001149
## b[6]  0.59194 0.11932 0.0009743      0.001774
## b[7]  0.27275 0.09941 0.0008117      0.001027
## b[8]  0.16882 0.10622 0.0008673      0.001454
## int  -0.86343 0.09713 0.0007930      0.001101
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## b[1]  0.20135  0.34053  0.40969  0.48108  0.61359
## b[2]  0.90495  1.04485  1.12388  1.20205  1.35999
## b[3] -0.30349 -0.17224 -0.10240 -0.03507  0.08939
## b[4] -0.16845 -0.02958  0.04127  0.11540  0.26248
## b[5] -0.23291 -0.10802 -0.04305  0.02050  0.14924
## b[6]  0.35898  0.50986  0.59056  0.67262  0.82955
## b[7]  0.08017  0.20469  0.27138  0.33933  0.46795
## b[8] -0.03275  0.09654  0.16754  0.23977  0.38140
## int  -1.05457 -0.92850 -0.86272 -0.79635 -0.67657
```

```
## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## b[1]           1      1.01
## b[2]           1      1.01
## b[3]           1      1.00
## b[4]           1      1.01
## b[5]           1      1.00
## b[6]           1      1.01
## b[7]           1      1.00
## b[8]           1      1.01
## int           1      1.00
##
## Multivariate psrf
##
## 1
```

```
## Mean deviance: 721.8
## penalty 8.804
## Penalized deviance: 730.6
```

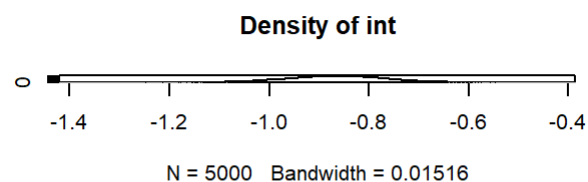
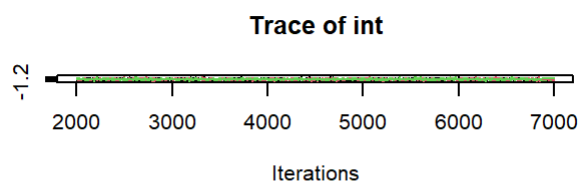
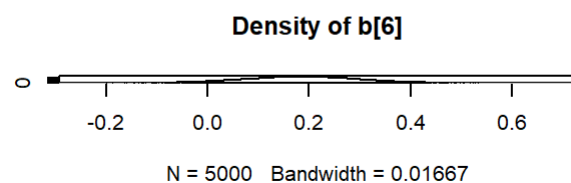
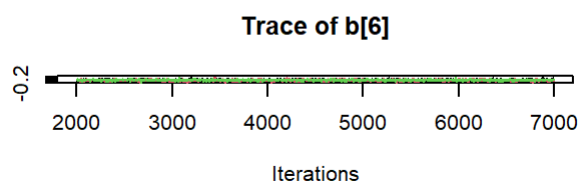
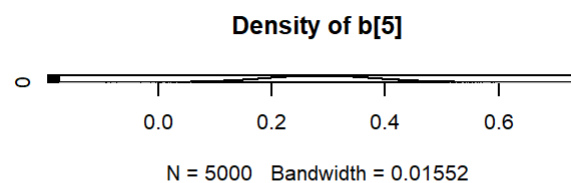
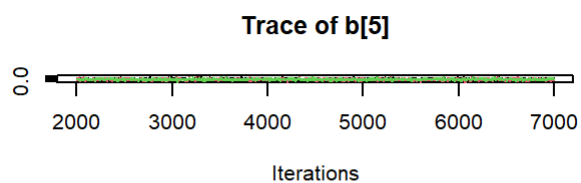
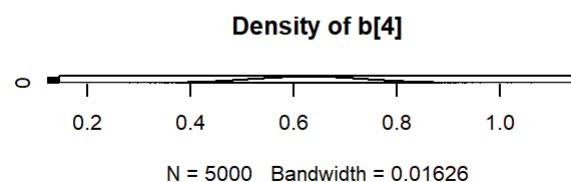
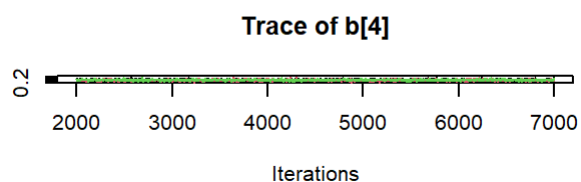
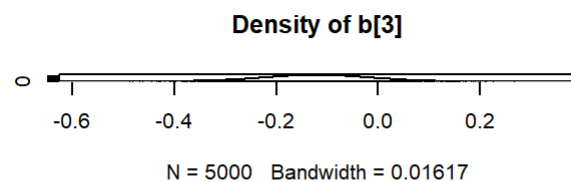
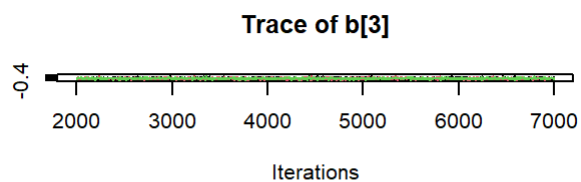
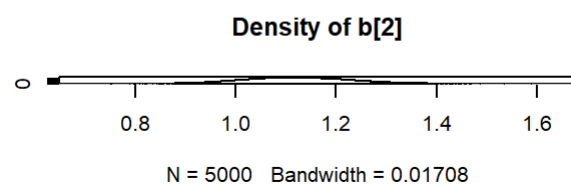
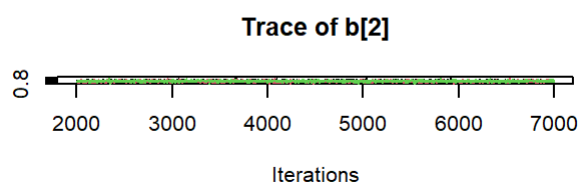
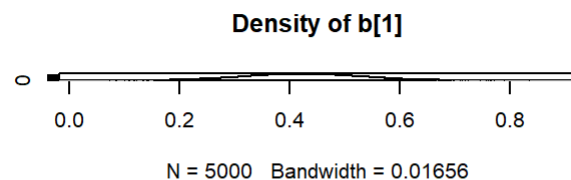
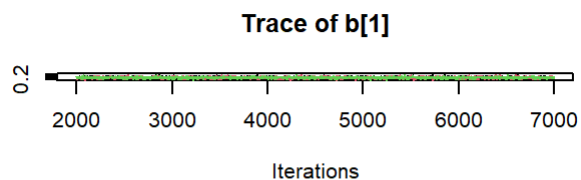
Model 2

- Ignoring `SkinThickness`, `Insulin` and refitting the model with flat normal prior (with zero mean, sd 5) on each of the predictor coefficients results in *DIC* of 739.5, a slight improvement from the previous model.
- The trace plots look okay, the Markov chains seem to have converged, with the autocorrelation plot supporting it. Effective sample sizes for the coefficients are quite large too.
- The model is used for prediction and it gives 77.35% accuracy.

```
mod2_string = " model {  
  for (i in 1:length(Outcome)) {  
    Outcome[i] ~ dbern(p[i])  
    logit(p[i]) = int + b[1]*Pregnancies[i] + b[2]*Glucose[i] + b[3]*BloodPressure[i] + b  
[4]*BMI[i]  
                      + b[5]*DiabetesPedigreeFunction[i] + b[6]*Age[i]  
  }  
  int ~ dnorm(0.0, 1.0/25.0)  
  for (j in 1:6) {  
    b[j] ~ dnorm(0.0, 1/25.0)  
  }  
}"  
  
mod2 = jags.model(textConnection(mod2_string), data=data_jags, n.chains=3)
```

```
## Compiling model graph  
##   Resolving undeclared variables  
##   Allocating nodes  
## Graph information:  
##   Observed stochastic nodes: 768  
##   Unobserved stochastic nodes: 7  
##   Total graph size: 7958  
##  
## Initializing model
```

```
update(mod2, 1e3)  
  
mod2_sim = coda.samples(model=mod2,  
                        variable.names=params,  
                        n.iter=5e3)  
mod2_csim = as.mcmc(do.call(rbind, mod2_sim))  
  
plot(mod2_sim, ask=TRUE)
```



```
summary(mod2_sim)
```

```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## b[1]  0.4235 0.10687 0.0008726      0.001403
## b[2]  1.1219 0.11029 0.0009005      0.001266
## b[3] -0.1225 0.10458 0.0008539      0.001282
## b[4]  0.6294 0.10497 0.0008571      0.001222
## b[5]  0.2859 0.10021 0.0008182      0.001047
## b[6]  0.1825 0.10763 0.0008788      0.001502
## int  -0.8676 0.09787 0.0007991      0.001087
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## b[1]  0.21721  0.3505  0.4239  0.49515  0.63455
## b[2]  0.91176  1.0460  1.1189  1.19478  1.34637
## b[3] -0.32939 -0.1918 -0.1227 -0.05194  0.07994
## b[4]  0.42705  0.5568  0.6280  0.70056  0.83654
## b[5]  0.08987  0.2177  0.2853  0.35256  0.48706
## b[6] -0.02793  0.1103  0.1828  0.25678  0.39102
## int  -1.06211 -0.9327 -0.8658 -0.80145 -0.67705
```

```
gelman.diag(mod2_sim)
```

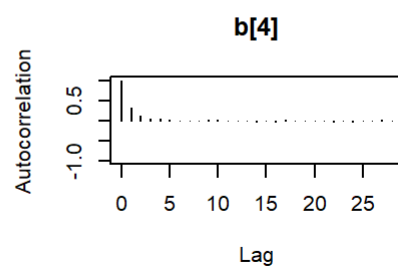
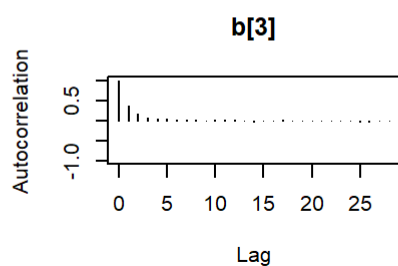
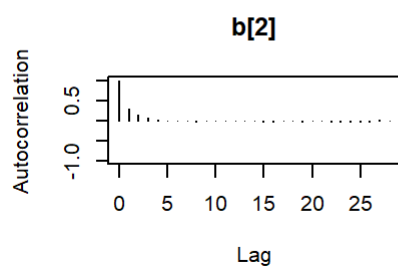
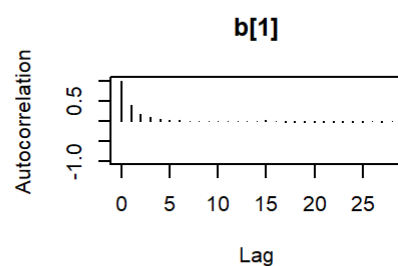
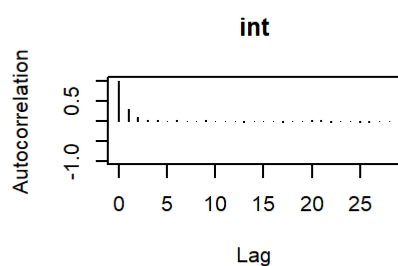
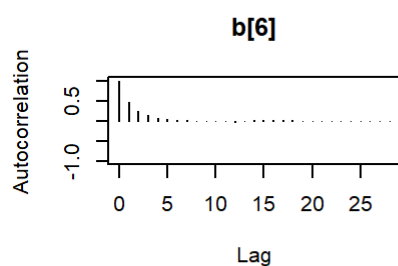
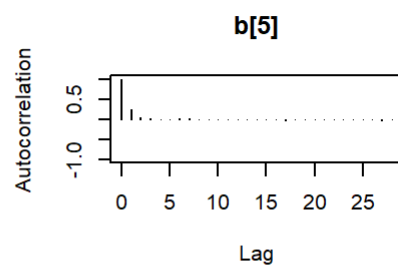
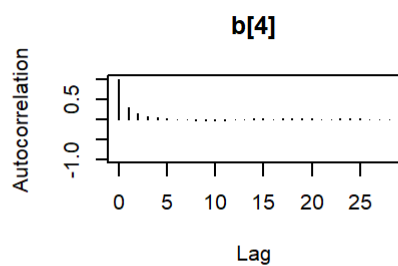
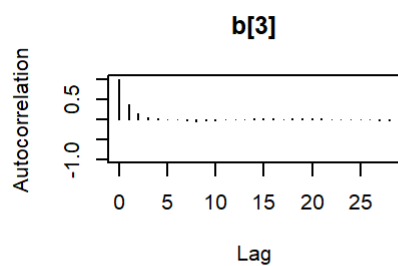
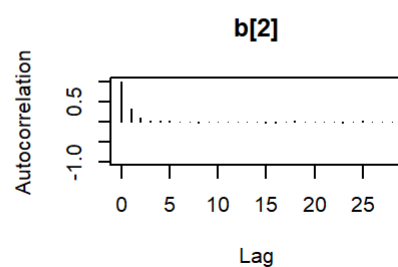
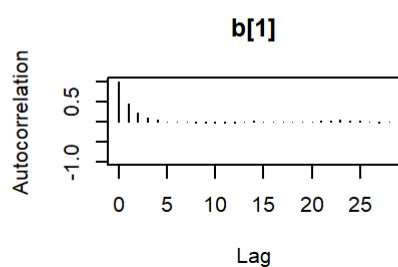
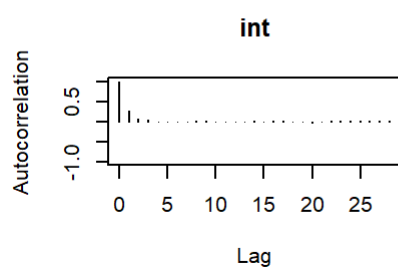
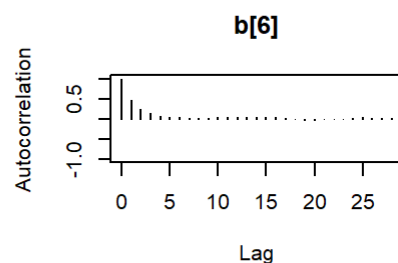
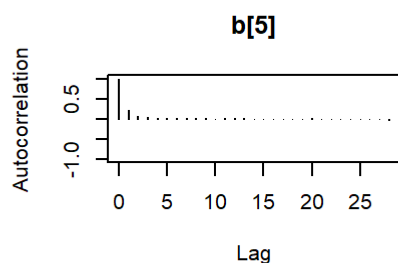
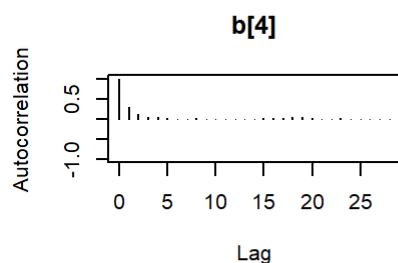
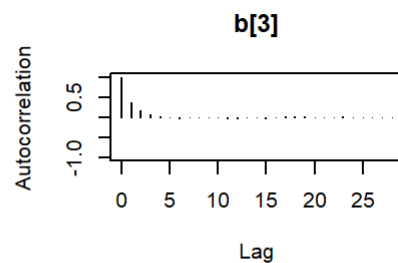
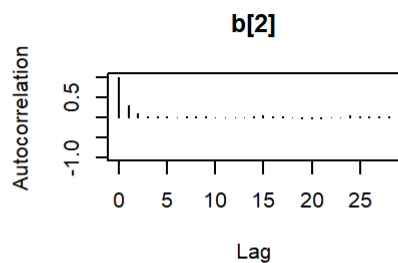
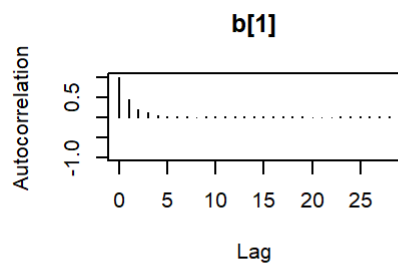
```
## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## b[1]           1      1.00
## b[2]           1      1.01
## b[3]           1      1.00
## b[4]           1      1.00
## b[5]           1      1.00
## b[6]           1      1.00
## int           1      1.01
##
## Multivariate psrf
##
## 1
```

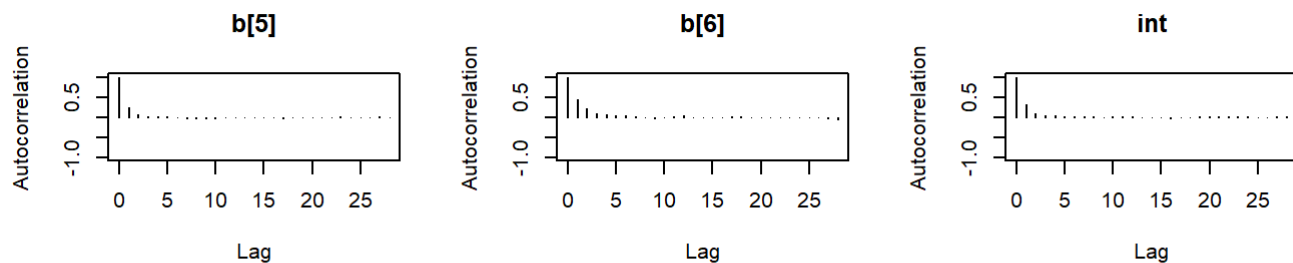
```
autocorr.diag(mod2_sim)
```



```
##          b[1]          b[2]          b[3]          b[4]          b[5]
## Lag 0  1.000000000  1.000000000  1.000000000  1.000000000  1.000000000
## Lag 1  0.422714906  0.298759471  0.369453175  0.306076857  0.228104782
## Lag 5  0.019086543  0.007765304  0.007945827  0.018445629  0.010524407
## Lag 10 0.005921652 -0.009432100 -0.005060211 -0.003797842 -0.006683677
## Lag 50 -0.007987130  0.002683121  0.003533049  0.012073507  0.001060634
##          b[6]          int
## Lag 0  1.000000000  1.000000000
## Lag 1  0.460948846  0.297754577
## Lag 5  0.046627198  0.006955801
## Lag 10 0.014868592  0.001500696
## Lag 50 -0.001517358  0.019522887
```

```
autocorr.plot(mod2_sim)
```





```
effectiveSize(mod2_sim)
```

```
##      b[1]      b[2]      b[3]      b[4]      b[5]      b[6]      int
## 5841.966 7609.825 6672.421 7380.122 9160.051 5149.071 8118.198
```

```
dic2 = dic.samples(mod2, n.iter=1e3)
dic2
```

```
## Mean deviance: 720.5
## penalty 7.054
## Penalized deviance: 727.5
```

```
(pm_coef = colMeans(mod2_csim))
```

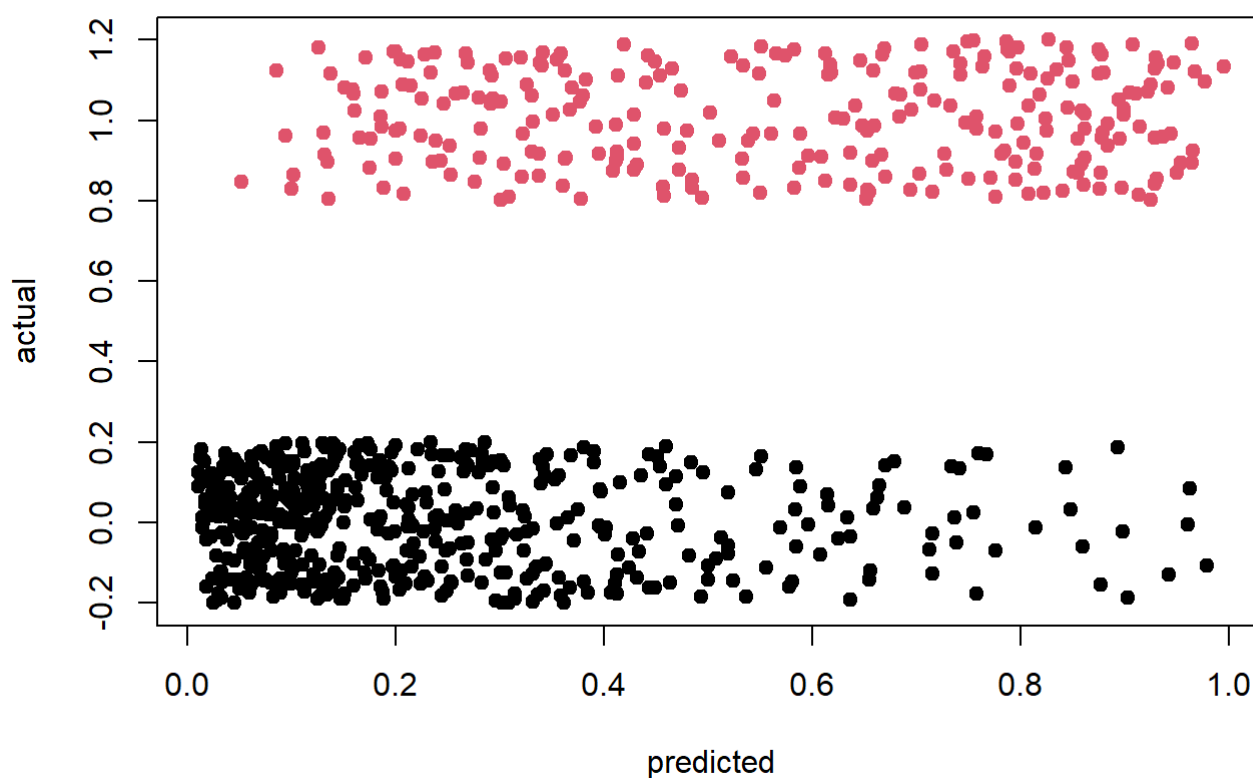
```
##      b[1]      b[2]      b[3]      b[4]      b[5]      b[6]      int
## 0.4234911 1.1218616 -0.1225443 0.6294060 0.2858975 0.1825130 -0.8675667
```

```
pm_Xb = pm_coef["int"] + as.matrix(X[,c(1,2,3,6,7,8)]) %*% pm_coef[1:6]
phat = 1.0 / (1.0 + exp(-pm_Xb))
head(phat)
```

```
##      [,1]
## 1 0.69407413
## 2 0.03463433
## 3 0.78441276
## 4 0.03367104
## 5 0.92843051
## 6 0.12639348
```

```
plot(phat, jitter(dat$Outcome), pch=19, col=dat$Outcome+1, xlab='predicted', ylab='actual', main='prediction with logistic regression')
```

prediction with logistic regression



```
(tab0.5 = table(phat > 0.5, data_jags$Outcome))
```

```
##
##      0    1
## FALSE 442 114
##  TRUE   58 154
```

```
sum(diag(tab0.5)) / sum(tab0.5)
```

```
## [1] 0.7760417
```

Model 3

- Using all eight of the predictors and refitting the model with flat normal prior (with zero mean, sd 5) on each of the predictor coefficients results in *DIC* of 741.3, a slight deterioration from the previous model.
- The trace plots look okay, the Markov chains seem to have converged, with the autocorrelation plot supporting it. Effective sample sizes for the coefficients are quite large too.
- The model is used for prediction and it gives 78.25% accuracy (on the same dataset it's trained on).

```
mod3_string = " model {
  for (i in 1:length(Outcome)) {
    Outcome[i] ~ dbern(p[i])
    logit(p[i]) = int + b[1]*Pregnancies[i] + b[2]*Glucose[i] + b[3]*BloodPressure[i] + b
[4]*SkinThickness[i] + b[5]*Insulin[i] + b[6]*BMI[i]
                  + b[7]*DiabetesPedigreeFunction[i] + b[8]*Age[i]
  }
  int ~ dnorm(0.0, 1.0/25.0)
  for (j in 1:8) {
    b[j] ~ dnorm(0.0, 1/25.0)
  }
} "
```

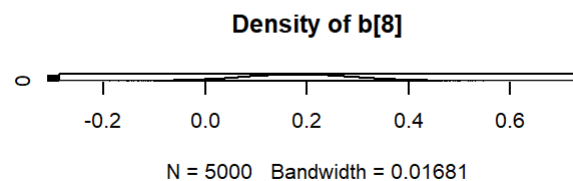
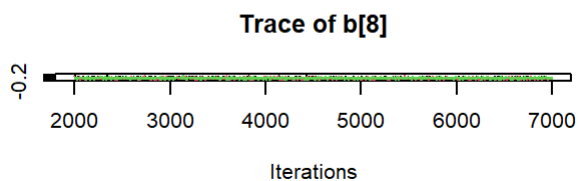
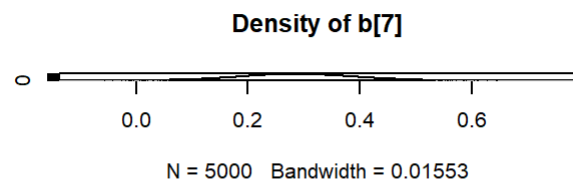
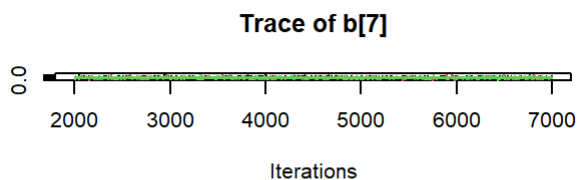
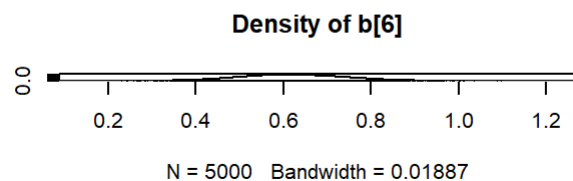
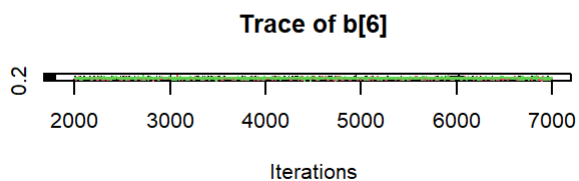
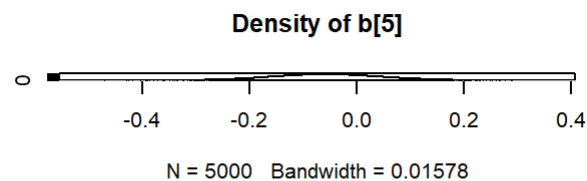
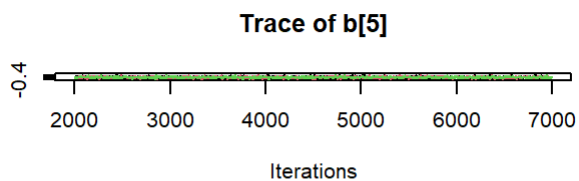
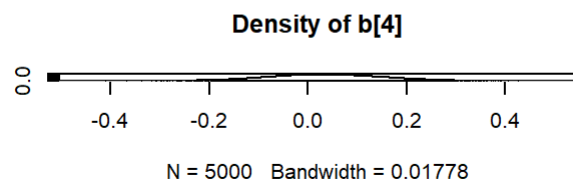
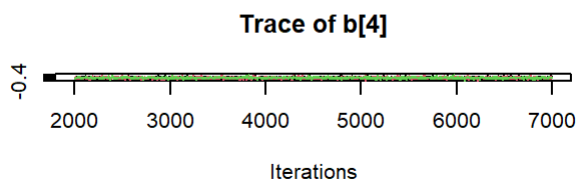
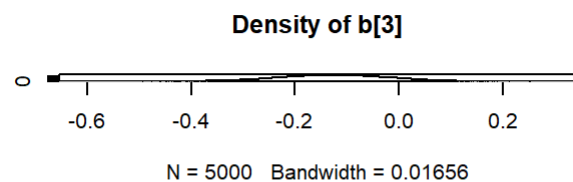
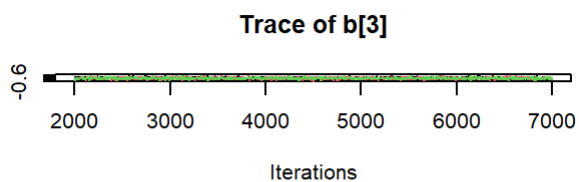
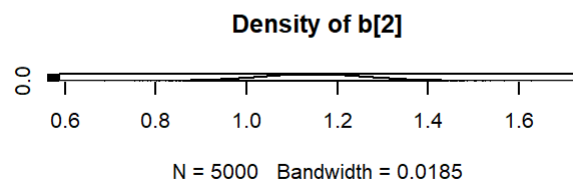
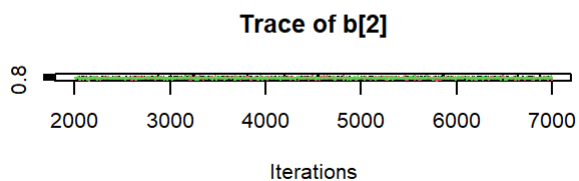
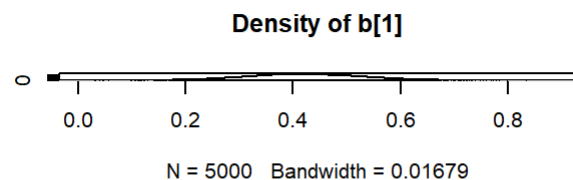
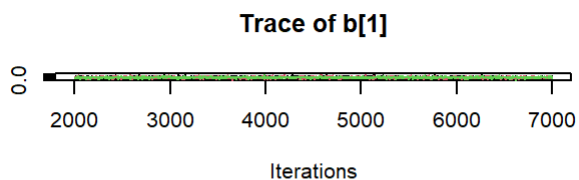
```
mod3 = jags.model(textConnection(mod3_string), data=as.list(X), n.chains=3)
```

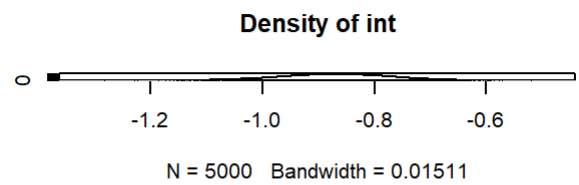
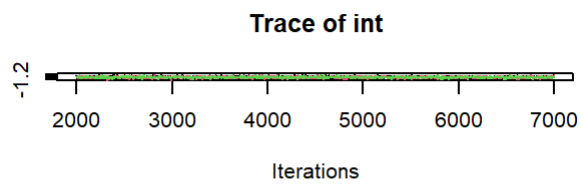
```
## Compiling model graph
##   Resolving undeclared variables
##   Allocating nodes
## Graph information:
##   Observed stochastic nodes: 768
##   Unobserved stochastic nodes: 9
##   Total graph size: 9739
##
## Initializing model
```

```
update(mod3, 1e3)

mod3_sim = coda.samples(model=mod3,
                        variable.names=params,
                        n.iter=5e3)
mod3_csim = as.mcmc(do.call(rbind, mod3_sim))

plot(mod3_sim, ask=TRUE)
```





```
summary(mod3_sim)
```

```
##
## Iterations = 2001:7000
## Thinning interval = 1
## Number of chains = 3
## Sample size per chain = 5000
##
## 1. Empirical mean and standard deviation for each variable,
##    plus standard error of the mean:
##
##           Mean      SD Naive SE Time-series SE
## b[1]  0.42136 0.10839 0.0008850      0.001457
## b[2]  1.14842 0.11945 0.0009753      0.001513
## b[3] -0.12796 0.10691 0.0008729      0.001369
## b[4]  0.03638 0.11475 0.0009370      0.001556
## b[5] -0.05620 0.10197 0.0008326      0.001211
## b[6]  0.62063 0.12181 0.0009946      0.001755
## b[7]  0.28839 0.10024 0.0008185      0.001049
## b[8]  0.18450 0.10854 0.0008862      0.001535
## int  -0.87118 0.09752 0.0007963      0.001082
##
## 2. Quantiles for each variable:
##
##           2.5%      25%      50%      75%      97.5%
## b[1]  0.21067  0.34812  0.41968  0.49436  0.63648
## b[2]  0.91934  1.06789  1.14657  1.22799  1.38841
## b[3] -0.34302 -0.19968 -0.12651 -0.05534  0.07561
## b[4] -0.18444 -0.04157  0.03433  0.11439  0.26298
## b[5] -0.25329 -0.12510 -0.05780  0.01136  0.14693
## b[6]  0.38637  0.53831  0.61784  0.70229  0.86368
## b[7]  0.09502  0.21974  0.28774  0.35570  0.48343
## b[8] -0.02621  0.11120  0.18336  0.25683  0.39846
## int  -1.06486 -0.93625 -0.87035 -0.80464 -0.68415
```

```
gelman.diag(mod3_sim)
```

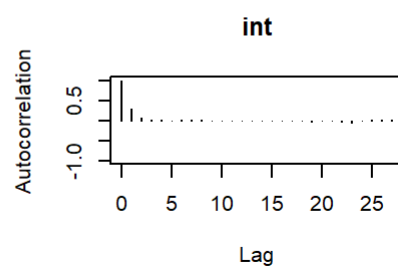
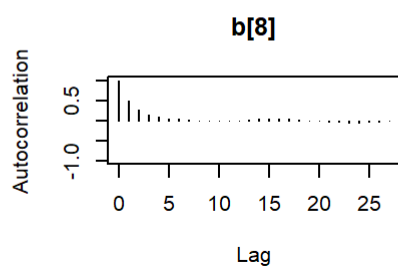
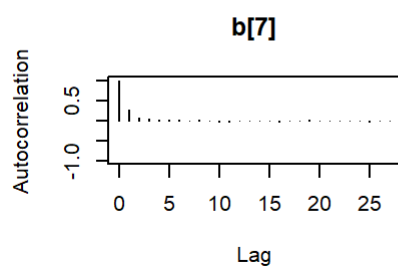
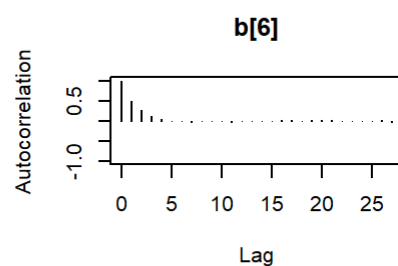
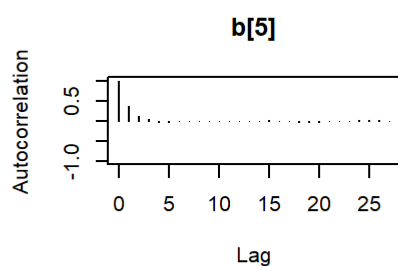
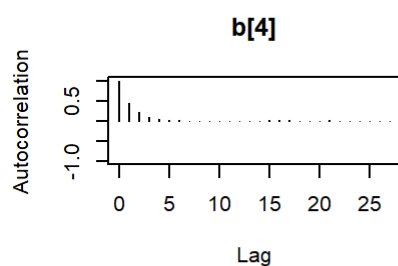
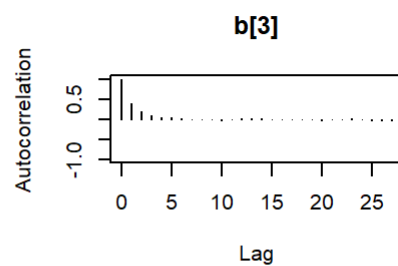
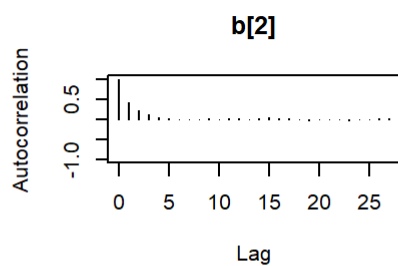
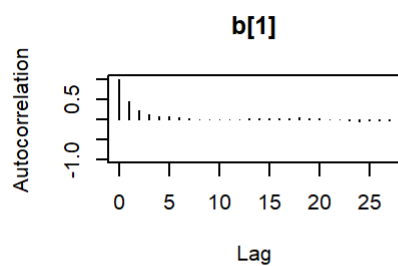
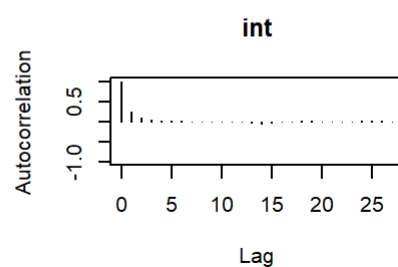
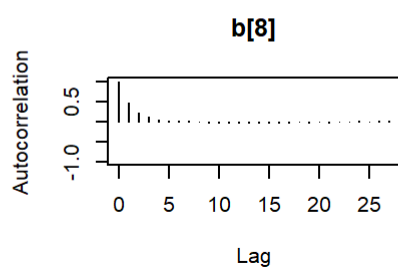
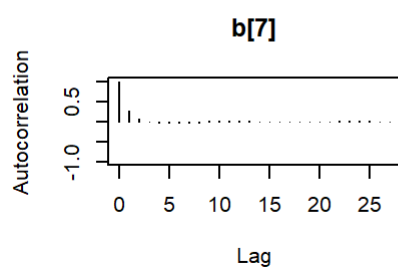
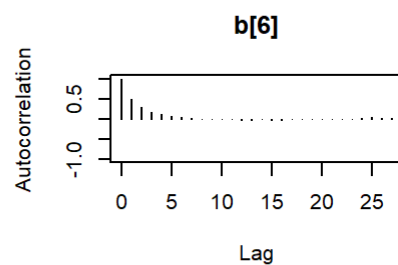
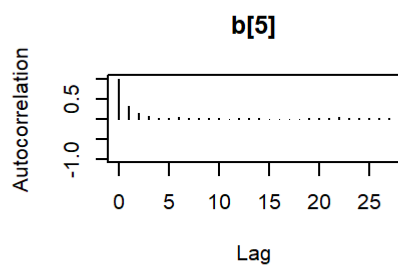
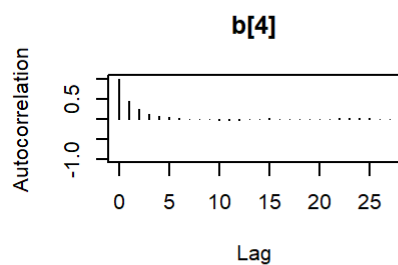
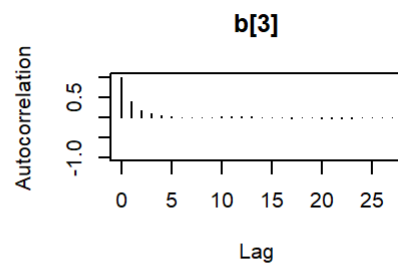
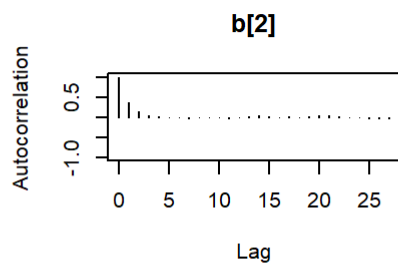
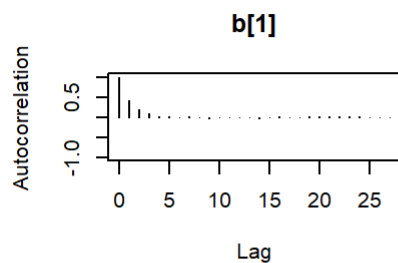
```
## Potential scale reduction factors:
##
##           Point est. Upper C.I.
## b[1]           1           1
## b[2]           1           1
## b[3]           1           1
## b[4]           1           1
## b[5]           1           1
## b[6]           1           1
## b[7]           1           1
## b[8]           1           1
## int           1           1
##
## Multivariate psrf
##
## 1
```

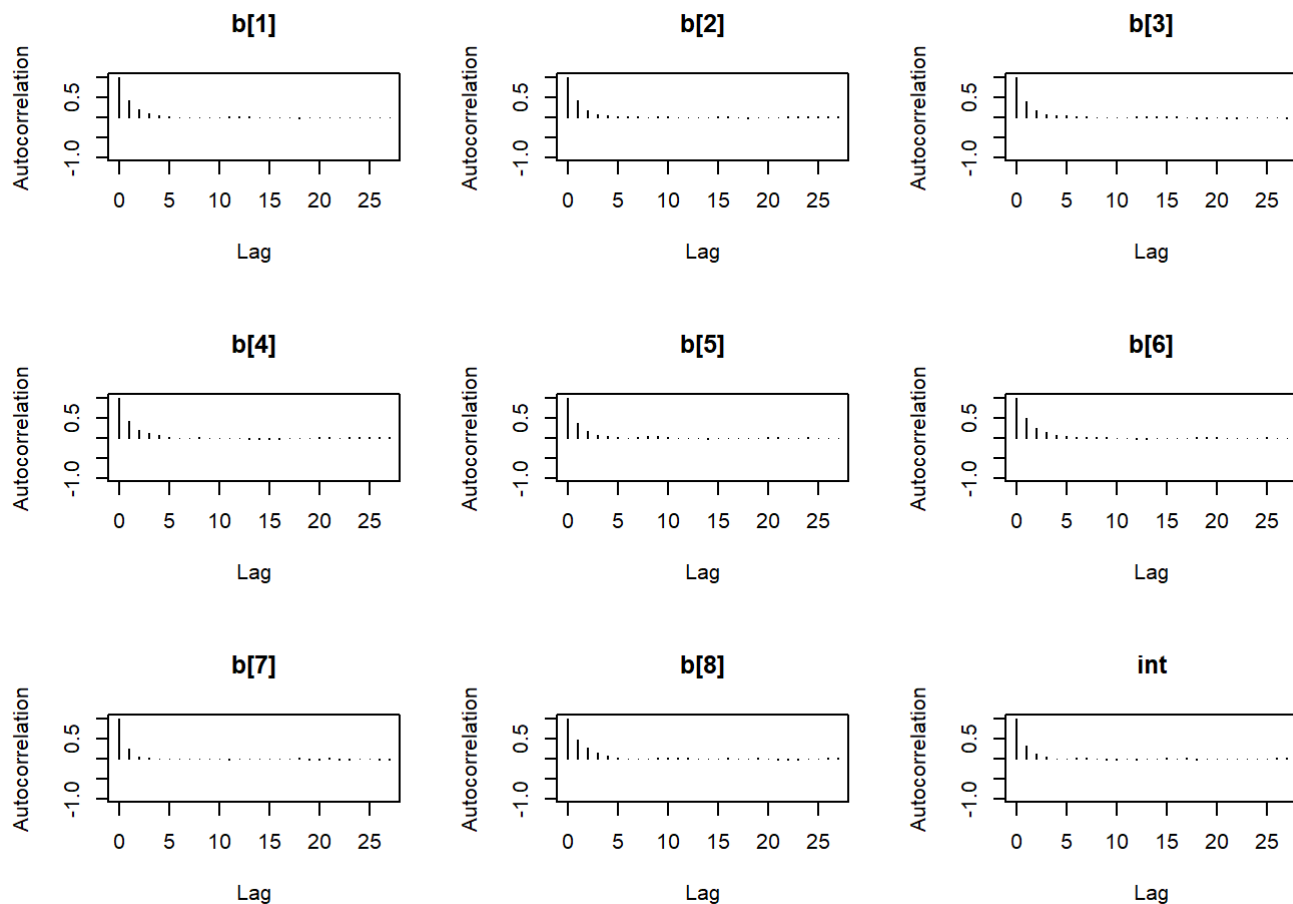
```
autocorr.diag(mod3_sim)
```


##	b[1]	b[2]	b[3]	b[4]	b[5]
## Lag 0	1.000000000	1.000000000	1.000000000	1.000000000	1.000000000
## Lag 1	0.423921042	0.399690898	0.399098732	0.43692510	0.350053771
## Lag 5	0.027102672	0.017786383	0.028509181	0.02645753	0.001925710
## Lag 10	-0.006635002	0.012007082	-0.001529713	-0.01001915	0.013356616
## Lag 50	0.009751964	-0.004338145	0.015515896	-0.01615248	0.002713069

##	b[6]	b[7]	b[8]	int
## Lag 0	1.000000000	1.000000000	1.000000000	1.000000000
## Lag 1	0.495448275	0.2643219901	0.4727381642	0.287042226
## Lag 5	0.033011348	-0.0071216337	0.0324494257	0.001509460
## Lag 10	-0.007655952	-0.0002218541	-0.0103685933	-0.011252856
## Lag 50	-0.006324225	0.0037074888	0.0006556728	0.002080019

```
autocorr.plot(mod3_sim)
```





```
effectiveSize(mod3_sim)
```

```
##      b[1]      b[2]      b[3]      b[4]      b[5]      b[6]      b[7]      b[8]
## 5561.674 6258.235 6102.400 5446.262 7127.696 4838.332 9183.143 5005.336
##      int
## 8131.978
```

```
dic3 = dic.samples(mod3, n.iter=1e3)
dic3
```

```
## Mean deviance: 722.2
## penalty 9.247
## Penalized deviance: 731.4
```

```
(pm_coef = colMeans(mod3_csim))
```

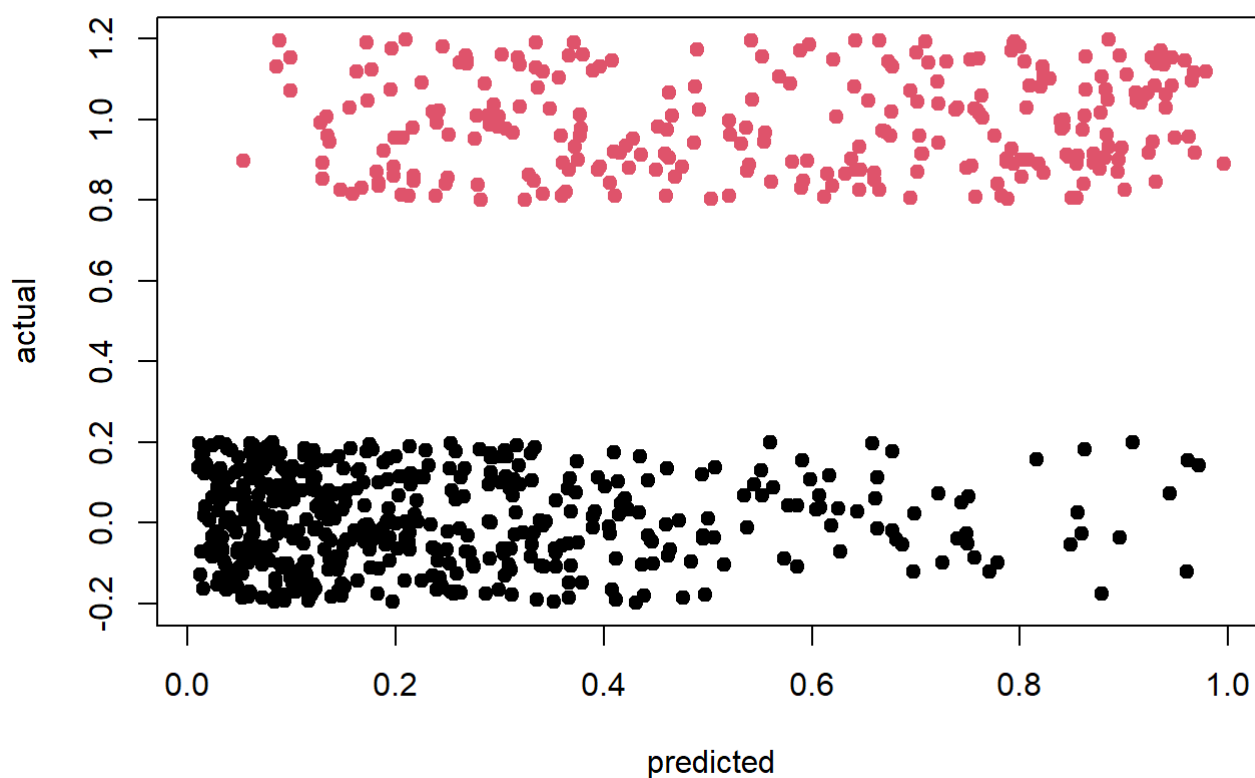
```
##      b[1]      b[2]      b[3]      b[4]      b[5]      b[6]
## 0.42136435 1.14841576 -0.12796373 0.03638261 -0.05619779 0.62063225
##      b[7]      b[8]      int
## 0.28838881 0.18450129 -0.87117596
```

```
pm_Xb = pm_coef["int"] + as.matrix(X[,1:8]) %*% pm_coef[1:8]
phat = 1.0 / (1.0 + exp(-pm_Xb))
head(phat)
```

```
##      [,1]
## 1 0.70358500
## 2 0.03381801
## 3 0.79505916
## 4 0.03333607
## 5 0.93106056
## 6 0.12600973
```

```
plot(phat, jitter(dat$Outcome), pch=19, col=dat$Outcome+1, xlab='predicted', ylab='actual', main='prediction with logistic regression')
```

prediction with logistic regression



```
(tab0.5 = table(phat > 0.5, data_jags$Outcome))
```

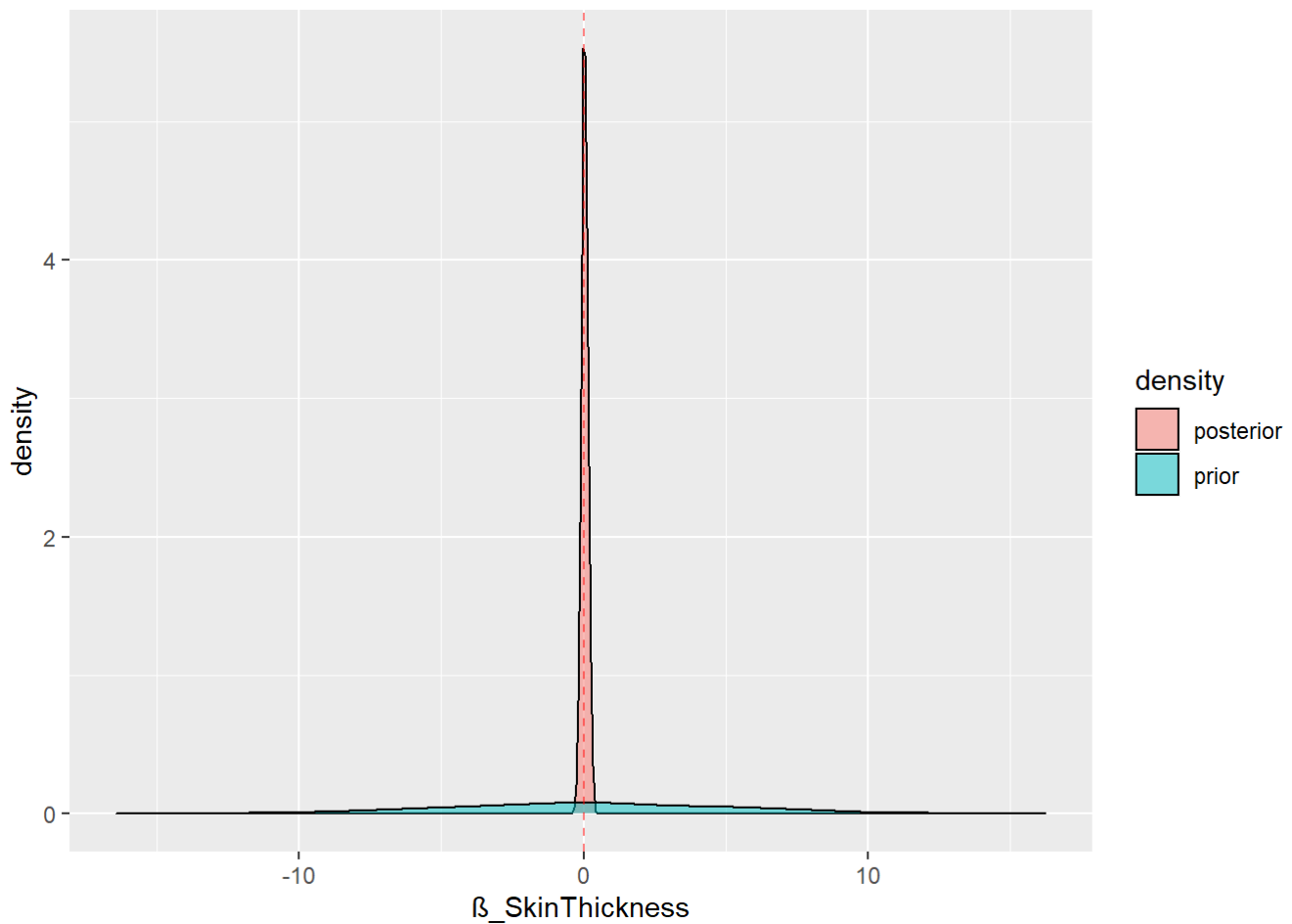
```
##
##      0    1
## FALSE 443 114
##  TRUE  57 154
```

```
sum(diag(tab0.5)) / sum(tab0.5)
```

```
## [1] 0.7773438
```

Prior vs. Posterior on Coefficients

The next plot shows how the flat priors compare with the sharp skinny posterior distribution for the coefficient for a predictor (samples drawn using MCMC simulation). Also it shows that the predictor `SkinThickness` is not an important predictor, since the posterior density includes zero.



Bayesian Credible Intervals (CI) for the Coefficients

The *Highest Posterior Density intervals* for the coefficients are shown below:

- e.g., the estimated coefficient for the predictor `Glucose` lies in the interval `[0.9089, 1.3892]` with 95% probability.

```
##          lower      upper
## b[1]  0.21011801  0.63532505
## b[2]  0.92446245  1.39304542
## b[3] -0.33678028  0.07936931
## b[4] -0.18846055  0.25570532
## b[5] -0.24997649  0.14962542
## b[6]  0.39607995  0.87225582
## b[7]  0.09428667  0.48243129
## b[8] -0.02639765  0.39809525
## int -1.06504145 -0.68461603
## attr("Probability")
## [1] 0.95
```

Make Probabilistic Statements

For example, let's try to answer the question: what's the probability that the effect size for `DiabetesPedigreeFunction` is greater than that of the predictor `Age` ? We can answer by conditioning on the samples obtained from simulation and then taking the mean as shown below: the probability is 0.824.

```
## [1] 0.7638667
```

Posterior Predictive Distribution

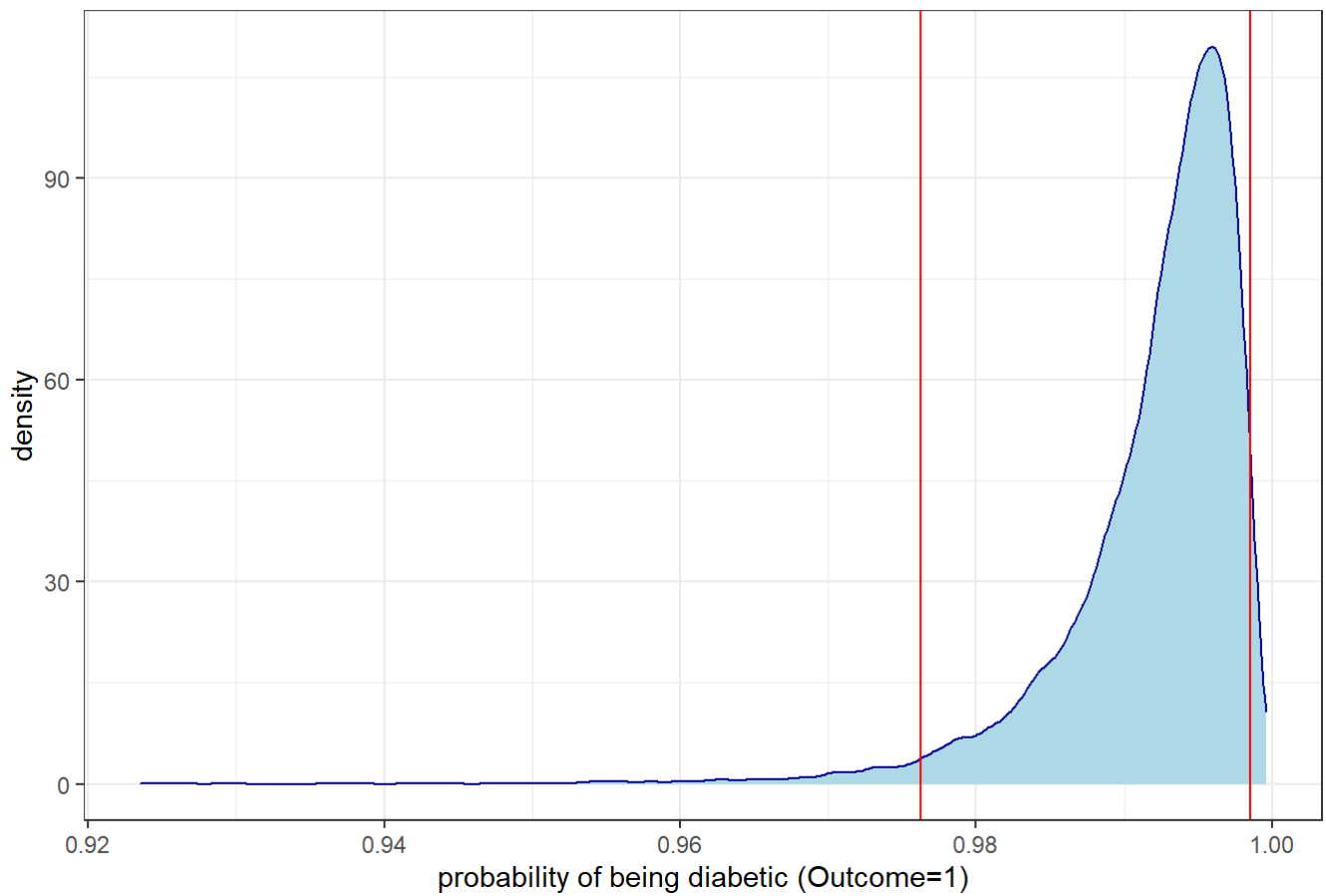
- Now, let's consider the following two new patients with the following diagnostic measurements for the predictors.
- First we need to scale the predictors by subtracting the mean and dividing by the s.d. of each of the respective predictors.
- Now, let's use the MCMC samples drawn to construct the posterior predictive distribution for patient one and construct 95% credible interval with *quantile*. Hence, with 95% of the time the probability that the first patient is diabetic will be in the interval [0.9708372, 0.9978117].
- Also, let's compare the probability that the second patient has higher probability of having diabetes than the first patient. As can be seen, it can also be computed from the MCMC samples and the result is 0.

```
##      Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin  BMI
## patient 1         0     300             80             30     100   33
## patient 2         0     250             82             29     90    35
##      DiabetesPedigreeFunction  Age
## patient 1                   0.30  40
## patient 2                   0.25  50
```

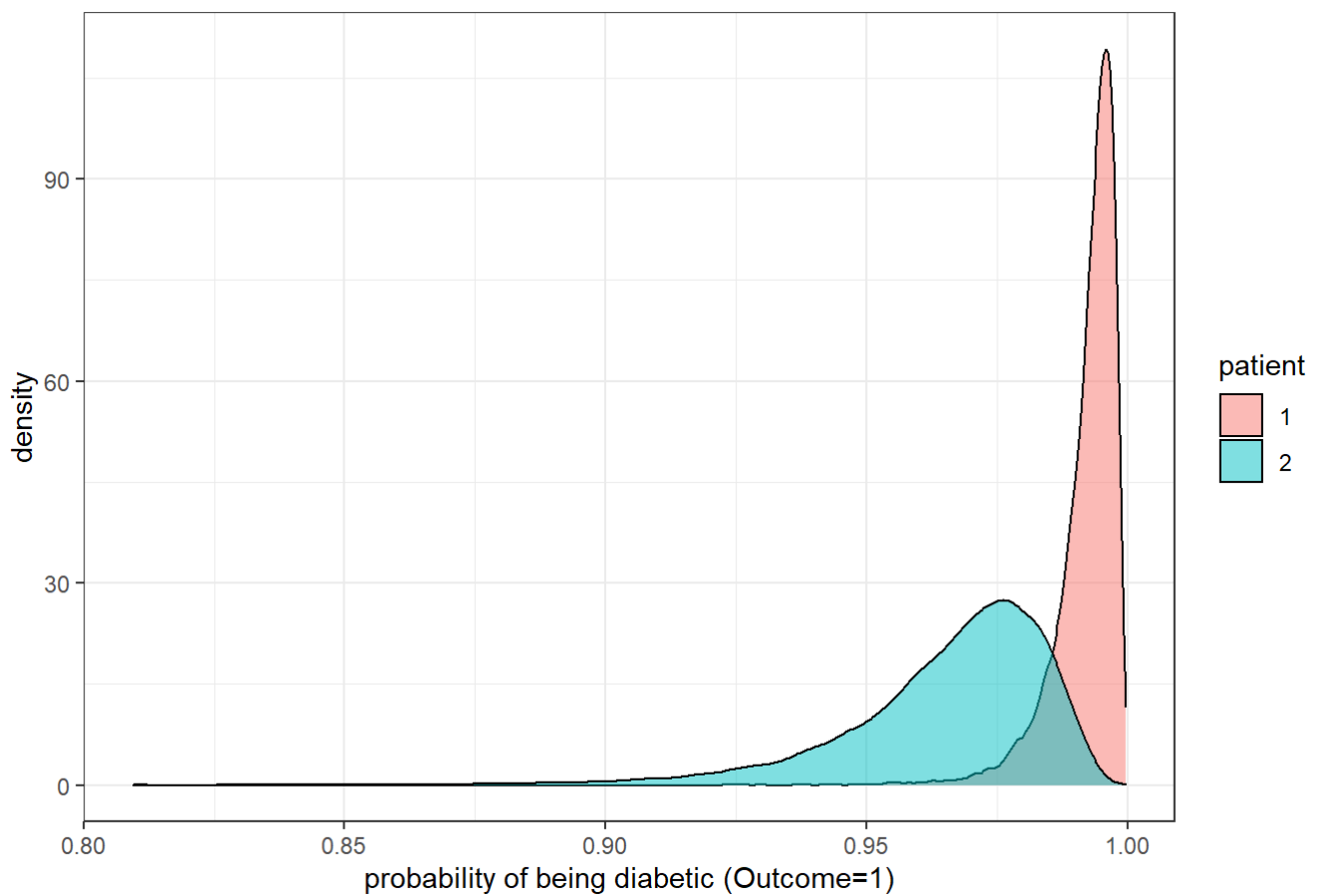
```
## Markov Chain Monte Carlo (MCMC) output:
## Start = 1
## End = 7
## Thinning interval = 1
##      [,1]
## [1,] 0.9977015
## [2,] 0.9985387
## [3,] 0.9989225
## [4,] 0.9752471
## [5,] 0.9937972
## [6,] 0.9951212
## [7,] 0.9884189
```

```
##      2.5%      97.5%
## 0.9762867 0.9985365
```

Posterior Predictive distribution 95% Credible Interval for patient 1



Posterior Predictive distribution for patient 1 and patient 2



```
## [1] 0
```

Conclusions

Using Bayesian Logistic Regression, we could not only compute the point estimates for the predictors and use them to predict the outcome, but the main strength of the model had been to quantify uncertainty in terms of how confident we are in terms of the estimates and predictions (using posterior predictive distribution). We can see that all predictor variables other than `Insulin` and `BloodPressure`, increase in their values increase the log odds of diabetes. The model even allows us to make probabilistic statements in terms of comparing two patients' probability of being diabetic, given the values of the diagnostic measurements of the predictor variables.

Improvements

The following can be tried to improve the model:

- Transform the predictors (e.g., with *log*) to remove skewness.
- Split the data into train and test partition, train the model on the train partition and predict on test partition.
- Use different imputation techniques, remove outliers.
- Using domain knowledge to pre/post process the predictors / extract features / do more feature engineering to improve accuracy.